

Backfocus Calculator

Manuel d'utilisation complet / Complete User Manual

Version 2.0 • 12 000+ produits • Bilingue EN/FR

*Application bilingue multiplateforme pour le calcul de backfocus en
astrophotographie.*

Bilingual cross-platform application for backfocus calculation in astrophotography.

Interface PyQt6 • Thème spatial sombre • Curseur galaxie • 125 marques • 22 types de
pièces

PyQt6 interface • Dark space theme • Galaxy cursor • 125 brands • 22 part types

Table des matières

I	Manuel en Français	2
1	Introduction	2
1.1	Qu'est-ce que le backfocus ?	2
1.2	Convention optique	2
1.3	Fonctionnalités principales	2
2	Installation	3
2.1	Prérequis	3
2.2	Sous Windows	3
2.3	Sous Linux	3
2.4	Sous macOS	3
2.5	Lancement direct	3
2.6	Exécutable Windows autonome (sans Python)	4
3	Catalogue de pièces	4
3.1	Premier lancement	4
3.2	Ajouter une pièce	4
3.3	Remplissage automatique (Auto-fill)	4
3.4	Modifier, dupliquer, supprimer	4
3.5	Quantité possédée	5
3.6	Filtres avancés	5
3.7	Tri par colonnes	5

4 Configurations	5
4.1 Créer une configuration	5
4.2 Gérer les configurations	5
4.3 Construire le train optique	5
4.4 Pièces fantômes	5
4.5 Zone de backfocus	6
4.6 Calcul automatique	6
4.7 Suggestion simple	6
4.8 Auto-complétion combinatoire	6
4.9 Diagramme visuel	6
5 Sauvegarde et Export	6
5.1 Sauvegarde automatique	6
5.2 Export / Import d'une configuration	7
5.3 Export / Import de toutes les données	7
6 Unités de mesure	7
7 Raccourcis clavier	7
8 Types de pièces	8
9 Types de connexion	9
10 Interface et thème	9
10.1 Thème spatial sombre	9
10.2 Curseur galaxie	9
10.3 Changement de langue	9
11 Structure des fichiers	9
12 Le concept de backfocus	10
13 Analyseur FITS / XISF de backfocus	10
13.1 Formats supportés	10
13.2 Principe de fonctionnement	11
13.3 Mosaïque 3×3	11
13.4 Interprétation	11
13.5 Dépendances	11
14 Rapports de bugs et capture de crashes	11
14.1 Capture automatique des crashes	11
14.2 Anonymisation	12
14.3 Détection au redémarrage	12
14.4 Rapport de bug manuel	12
15 Mises à jour automatiques	12
15.1 Vérification automatique	12
15.2 Vérification manuelle	12
15.3 Sécurité du téléchargement	12
16 Raccourci bureau et multi-PC	12
16.1 Création du raccourci	12
16.2 Architecture multi-PC / NAS	13

17 Performance et fluidité (v2.0)	13
17.1 Interface PyQt6 native	13
17.2 Sauvegarde asynchrone	13
17.3 Curseur galaxie optimisé	13
17.4 Catalogue : mise à jour rapide des quantités	13
17.5 Cache de recherche pièces	13
17.6 Débounce unifié	13
 II English Manual	 14
18 Introduction	14
18.1 What is Backfocus?	14
18.2 Optical Convention	14
18.3 Key Features	14
19 Installation	15
19.1 Requirements	15
19.2 Windows	15
19.3 Linux	15
19.4 macOS	15
19.5 Direct Launch	15
19.6 Windows Standalone (.exe) — No Python Required	15
20 Parts Catalog	16
20.1 First Launch	16
20.2 Adding a Part	16
20.3 Auto-fill from Reference Database	16
20.4 Editing, Duplicating, Deleting	16
20.5 Owned Quantity	16
20.6 Advanced Filters	16
20.7 Column Sorting	17
21 Configurations	17
21.1 Creating a Configuration	17
21.2 Managing Configurations	17
21.3 Building the Optical Train	17
21.4 Ghost Pieces	17
21.5 Backfocus Zone	18
21.6 Automatic Calculation	18
21.7 Simple Suggestion	18
21.8 Combinatorial Auto-complete	18
21.9 Visual Diagram	18
22 Save and Export	18
22.1 Automatic Save	18
22.2 Configuration Export / Import	18
22.3 Full Data Export / Import	18
23 Measurement Units	19
24 Keyboard Shortcuts	19

25 Part Types	19
26 Connection Types	20
27 Interface and Theme	20
27.1 Dark Space Theme	20
27.2 Galaxy Cursor	20
27.3 Language Switching	20
28 Project Structure	20
29 The Backfocus Concept	21
30 FITS / XISF Backfocus Analyzer	21
30.1 Supported Formats	21
30.2 How It Works	21
30.3 Mosaic 3×3	22
30.4 Interpretation	22
30.5 Dependencies	22
31 Bug Reports and Crash Capture	22
31.1 Automatic Crash Capture	22
31.2 Anonymization	23
31.3 Crash Detection on Restart	23
31.4 Manual Bug Report	23
32 Automatic Updates	23
32.1 Automatic Check	23
32.2 Manual Check	23
32.3 Download Security	23
33 Desktop Shortcut and Multi-PC	23
33.1 Creating the Shortcut	23
33.2 Multi-PC / NAS Architecture	24
34 Performance & Fluidity (v2.0)	24
34.1 Native PyQt6 Interface	24
34.2 Asynchronous Save	24
34.3 Galaxy Cursor Optimization	24
34.4 Catalog: Fast Quantity Update	24
34.5 Parts Search Cache	24
34.6 Unified Debounce	24

Première partie

Manuel en Français

Introduction

Le **Backfocus Calculator v2.0** est une application graphique de bureau basée sur **PyQt6** permettant de calculer et gérer les configurations de backfocus dans les trains optiques d'astrophotographie.

Qu'est-ce que le backfocus ?

En astrophotographie, le **backfocus** est la distance optique précise requise entre le dernier élément optique correcteur (aplanisseur, réducteur de focale, correcteur de coma) et le capteur de la caméra. Un backfocus incorrect provoque :

- Des étoiles déformées (coma, astigmatisme)
- Un champ courbé (étoiles nettes au centre, floues aux bords)
- Un tilt optique
- Un vignettage asymétrique

Chaque correcteur/réducteur est conçu pour une distance de backfocus spécifique (typiquement 55 mm, mais cela varie de 17 mm à 105 mm selon les modèles).

Convention optique

La lumière va de **gauche à droite** dans toute l'application :

- **Côté télescope** (gauche) : côté par lequel la lumière entre
- **Côté caméra** (droite) : côté par lequel la lumière sort vers le capteur

Chaque pièce possède une connexion **côté télescope** et une connexion **côté caméra**, chacune avec un diamètre de filetage et un genre (mâle = externe, femelle = interne).

Fonctionnalités principales

- Base de référence intégrée de **12 000+ produits réels** (125 marques)
- Catalogue de pièces avec **22 types** d'équipements
- **Remplissage automatique** des champs depuis la base de référence
- Construction de trains optiques avec **vérification de compatibilité** des connexions
- Zone de backfocus définie par l'utilisateur (marqueurs début/fin)
- **Retourner** les pièces réversibles (échange côté T/côté C)
- **Suggestion** et **auto-complétion** combinatoire (1 à 3 pièces)
- Détection de **conflits** basée sur les quantités possédées
- Diagramme visuel coloré avec zone de backfocus en surbrillance
- **29 types de connexion** : M42-M117, EOS, Canon RF, Nikon F/Z, Sony E, Fuji X, MFT, SC, vis ZWO/QHY
- Unités de mesure : mm/pouces, g/onces
- **Sauvegarde automatique** à la fermeture
- **Export/Import** de configurations individuelles ou de **toutes les données**
- Interface bilingue français/anglais
- Thème sombre spatial avec curseur galaxie animé
- **Mises à jour automatiques** via GitHub Releases

- **Rapports de bugs anonymisés** (crash auto + rapport manuel)
- **Raccourci bureau** créé depuis le menu Aide

Installation

Prérequis

- **Python 3.8** ou supérieur
- **PyQt6** (installé automatiquement par les scripts de lancement)
- **Aucune installation manuelle** requise

Sous Windows

Double-cliquez sur `launch.bat`. Le script :

1. Détecte Python sur votre système
2. Crée un environnement virtuel **local** (dans `%LOCALAPPDATA%\BackfocusCalculator\venv`)
3. Installe toutes les dépendances depuis `requirements.txt`
4. Lance l'application

Multi-PC / NAS

L'environnement virtuel est stocké **localement sur chaque PC**, pas dans le dossier du projet. Le dossier projet peut être sur un NAS ou un disque synchronisé sans aucun conflit.

Sous Linux

Debian/Ubuntu

```
sudo apt-get install python3 python3-venv python3-pip
```

Fedora

```
sudo dnf install python3 python3-pip
```

Arch

```
sudo pacman -S python
```

Lancer

```
chmod +x launch.sh && ./launch.sh
```

Le venv est créé dans `~/.local/share/BackfocusCalculator/venv`.

Sous macOS

```
brew install python
```

```
chmod +x launch.sh && ./launch.sh
```

Lancement direct

```
pip install PyQt6
```

```
python backfocus.py
```

Exécutable Windows autonome (sans Python)

Un exécutable Windows prêt à l'emploi est disponible sur la page [Releases](#) du projet :

1. Téléchargez le dernier `BackfocusCalculator-vX.X.X.zip`
2. Décompressez l'archive
3. Lancez `BackfocusCalculator.exe` — aucune installation de Python requise

Pour compiler l'exécutable vous-même :

```
pip install pyinstaller
pyinstaller backfocus.spec
```

Le résultat se trouve dans `dist/BackfocusCalculator/`. Toutes les fonctionnalités sont identiques.

Note : l'exécutable autonome est réservé à Windows. Sur Linux/macOS, utilisez les scripts de lancement ci-dessus.

Catalogue de pièces

Le catalogue s'ouvre dans une **grande fenêtre séparée** via le menu **Affichage > Catalogue de pièces** ou le bouton dans la barre d'outils.

Premier lancement

Au premier lancement, la base complète de 12 000+ produits est automatiquement chargée dans votre catalogue avec une quantité possédée de 0.

Ajouter une pièce

Cliquez sur **Ajouter** et remplissez :

- **Marque** — liste déroulante avec 125+ marques + option « Custom Made »
- **Nom** — nom du produit
- **Type** — parmi 22 catégories (voir Section 8)
- **Longueur optique** — contribution au backfocus en mm
- **Masse** — en grammes
- **Connexion côté télescope** — filetage + genre (Mâle/Femelle)
- **Connexion côté caméra** — filetage + genre
- **Réversible** — cocher si la pièce peut être retournée
- **Rôle backfocus** — **start**, **end** ou vide
- **Quantité** — nombre d'exemplaires possédés
- **Notes** — texte libre

Remplissage automatique (Auto-fill)

1. Tapez le début de la marque ou du nom du produit
2. Cliquez le bouton **Auto-fill**
3. Tous les champs sont remplis automatiquement depuis la base de 12 000+ produits

Modifier, dupliquer, supprimer

- **Double-cliquez** sur une pièce pour la modifier
- **Dupliquer** crée une copie (suffixe « (copy) »)
- **Supprimer** demande une confirmation

Quantité possédée

Les boutons + et – ajustent rapidement la quantité :

- Quantité > 0 ⇒ pièce affichée en **doré** (possédée)
- Quantité = 0 ⇒ pièce affichée en **gris** (non possédée)

Filtres avancés

Cinq filtres se combinent (logique ET) :

1. **Texte** — recherche dans la marque et le nom
2. **Type** — filtrer par catégorie
3. **Diamètre** — filtrer par filetage (M42, M48, M54, M68, SC, EOS, etc.)
4. **Genre** — Mâle, Femelle ou Tous
5. **Possédées uniquement** — cocher pour n’afficher que les pièces possédées

Bouton **Réinit.** pour effacer tous les filtres.

Tri par colonnes

Cliquez sur n’importe quel en-tête de colonne pour trier. Un deuxième clic inverse l’ordre.

Configurations

La fenêtre principale est dédiée aux configurations de trains optiques.

Créer une configuration

1. Cliquez + dans le panneau de gauche
2. Donnez un nom (ex : « Mon Newton + ASI 2600 »)
3. Indiquez le **backfocus cible** en mm (ex : 55)
4. Ajoutez des **notes** si besoin

Gérer les configurations

- – : supprimer (avec confirmation)
- **Renommer** : changer le nom
- **Dupliquer** : créer une copie complète

Construire le train optique

1. Cliquez **Ajouter pièce** pour choisir une pièce du catalogue
2. Les pièces s’ajoutent dans l’ordre télescope → caméra
3. **Haut/Bas** : réordonner les pièces
4. **Retourner** : retourner une pièce réversible
5. **Retirer** : enlever une pièce du train

Pièces fantômes

Quand vous ajoutez une pièce dont les connexions sont incompatibles avec la pièce précédente (par exemple un filetage M48 après un M42), l’application propose plusieurs options :

- **Retourner et insérer** : retourne la pièce si elle est réversible
- **Insérer quand même** : ignore l’incompatibilité
- **Insérer un fantôme** : insère un *adaptateur fantôme* (placeholder) entre les deux pièces
- **Modifier la pièce** : ouvre l’éditeur pour corriger les connexions

Le fantôme apparaît en **orange** dans le train avec le nom « ? Adaptateur manquant ». Il représente une bague d'adaptation que vous devez trouver. Ses connexions sont automatiquement déduites des pièces voisines.

Pour résoudre un fantôme :

1. Cliquez **Résoudre fantômes** dans la barre d'outils
2. L'application recherche dans votre catalogue les pièces compatibles avec les connexions requises
3. Sélectionnez une pièce et validez : elle **remplace le fantôme** à sa position exacte dans le train

Vous pouvez aussi insérer manuellement un fantôme avec le bouton **Insérer fantôme** pour marquer un emplacement à combler.

Zone de backfocus

1. Sélectionnez la pièce « début » (ex : réducteur) → **Début BF**
2. Sélectionnez la pièce « fin » (ex : caméra) → **Fin BF**
3. BF calculé = somme des longueurs optiques **après** le marqueur de début jusqu'au marqueur de fin (le début sert de point zéro, sa longueur n'est pas comptée)

Calcul automatique

L'application calcule en temps réel :

- **Total** : longueur optique totale du train
- **Backfocus** : longueur dans la zone BF uniquement
- **Écart** :
 - **Vert / OK** : écart < 0.1 mm
 - **Orange / Court** : backfocus insuffisant
 - **Rouge / Long** : backfocus excessif
- **Compatibilité** : vérifie entre chaque paire de pièces adjacentes
- **Conflits** : avertit si une pièce dépasse la quantité possédée

Suggestion simple

Suggérer pièce : cherche UNE pièce possédée qui comblerait l'écart. Vérifie la compatibilité de connexion. Résultats triés par proximité.

Auto-complétion combinatoire

Auto-compléter : recherche parmi les combinaisons de 1, 2 ou 3 pièces possédées. Option pour autoriser/interdire les pièces d'autres configurations. Solutions parfaites en **vert**.

Diagramme visuel

Diagramme en bas de l'écran : télescope à gauche, caméra à droite. Chaque couleur correspond à un type de pièce. Zone BF en surbrillance. Ligne pointillée rouge = cible. Glisser-déposer pour réordonner les pièces.

Sauvegarde et Export

Sauvegarde automatique

- Sauvegarde automatique dans `backfocus_data.json` à chaque fermeture
- Menu **Fichier** > **Tout enregistrer** pour sauvegarder manuellement

Export / Import d'une configuration

- **Fichier > Exporter la configuration** — JSON
- **Fichier > Importer la configuration** — depuis un JSON

Export / Import de toutes les données

- **Fichier > Exporter toutes les données** — TOUT : pièces, configs, réglages
- **Fichier > Importer toutes les données** — remplace TOUT (avec confirmation)
- Idéal pour transfert entre ordinateurs ou sauvegarde complète

Unités de mesure

Menu **Paramètres > Unités de mesure** :

- Longueurs : **mm** ou **pouces**
- Masse : **grammes** ou **onces**

Raccourcis clavier

Raccourci	Action
Entrée	Confirmer dans toutes les fenêtres de dialogue
Échap	Fermer / annuler toutes les fenêtres de dialogue
Double-clic (catalogue)	Modifier la pièce sélectionnée
Double-clic (suggestions)	Insérer la pièce ou solution

Types de pièces

Clé interne	Français	Rôle BF
type_telescope	Télescope	—
type_refractor	Lunette	—
type_camera_lens	Objectif photo	—
type_camera	Caméra astro	<i>end</i>
type_dslr	Reflex / Hybride	<i>end</i>
type_eyepiece	Oculaire	<i>end</i>
type_barlow	Barlow	<i>start</i>
type_reducer	Réducteur	<i>start</i>
type_flattener	Aplanisseur	<i>start</i>
type_extender	Extenseur	<i>start</i>
type_corrector	Correcteur	<i>start</i>
type_filter_wheel	Roue à filtres	—
type_filter_holder	Porte-filtre	—
type_oag	OAG	—
type_rotator	Rotateur	—
type_focuser	Porte-oculaire	—
type_diagonal	Renvoi coudé	—
type_adapter	Bague d'adaptation	—
type_spacer	Espaceur	—
type_anti_tilt	Anti-tilt	—
type_guide_scope	Lunette guide	—
type_flip_mirror	Miroir basculant	—

Types de connexion

Type	Description	Diamètre (mm)
M42 / T2	Filetage T2 (M42×0.75)	42
M48	Filetage filtre 2"	48
M54	Filetage ZWO courant	54
M56	Filetage caméra spécifique	56
M63	Filetage adaptateur moyen	63
M68	Filetage grand format	68
M72	Filetage aplanisseur/réducteur	72
M81	Filetage grand porte-oculaire	81
M82	Filetage très grand	82
M84	Filetage extra-large	84
M92	Filetage spécialisé	92
M117	Filetage très grand format	117
SC	Baïonnette Schmidt-Cassegrain	—
EOS	Monture Canon EOS (EF)	54
Canon RF	Monture Canon RF	54
Nikon F	Monture Nikon F	44
Nikon Z	Monture Nikon Z	55
Sony E	Monture Sony E	46
Fuji X	Monture Fujifilm X	44
MFT	Monture Micro Four Thirds	38
Pentax K	Monture Pentax K	44
CS	Monture C/CS	25.4
1.25"	Coulant oculaire 31.75 mm	31.75
2"	Coulant oculaire 50.8 mm	50.8
ZWO 6-bolt	Fixation 6 vis ZWO	—
ZWO 4-bolt	Fixation 4 vis ZWO	—
QHY 4-bolt	Fixation 4 vis QHY	—

Interface et thème

Thème spatial sombre

L'application utilise un thème sombre inspiré de l'espace : fond noir/bleu nuit avec accents colorés, pièces possédées en doré, non-possédées en gris. L'interface est entièrement en **PyQt6** avec un style QSS intégré.

Curseur galaxie

Un curseur en forme de galaxie spirale suit votre souris. Il est purement décoratif.

Changement de langue

Menu **Langue** → **Français** ou **English**. Instantané et mémorisé.

Structure des fichiers

backfocus/

- backfocus.py	Application principale (PyQt6, ~3400 lignes)
- reference_data.py	Base de données (12 100+ entrées)
- gen_refdb.py	Générateur de la base de données
- backfocus_data.json	Données utilisateur (auto-crée)
- fits_analyzer.py	Analyseur FITS/XISF de backfocus (optionnel)
- test_audit.py	Suite de tests automatisés
- shortcut_helper.py	Aide création raccourci bureau
- launch.bat	Lanceur Windows (venv local)
- launch.sh	Lanceur Linux/macOS (venv local)
- requirements.txt	Dependances Python (PyQt6 + analyseur FITS)
- backfocus.spec	Spec PyInstaller (exe Windows)
- backfocus.ico	Icône application (Windows)
- backfocus.png	Icône application (multiplateforme)
- README.md	Documentation GitHub
- manual/	
' - manual.pdf	Manuel bilingue (PDF)

Le concept de backfocus

Le **backfocus** se mesure de la face caméra du correcteur/réducteur/aplanisseur jusqu'au capteur.

Exemples de backfocus courants :

Correcteur / Réducteur	BF requis
Sky-Watcher 0.85x Reducer	55 mm
Baader MPCC Mark III	55 mm
Celestron f/6.3 Reducer	99 mm
Starizona HyperStar	12–24 mm
Takahashi TOA-35 Reducer	72.2 mm
Riccardi 0.75x M82 Reducer	56 mm
ASA 3" Corrector	67 mm

Certains télescopes (ex : réfracteurs Petzval comme le Takahashi FSQ-106ED) n'ont pas de concept de backfocus séparé. L'application supporte ces cas en ne définissant simplement pas de zone BF.

Analyseur FITS / XISF de backfocus

L'application inclut un analyseur d'images qui diagnostique les erreurs de backfocus à partir d'une seule image astro. Ouvrez-le via le menu **Affichage > Analyseur FITS / XISF de backfocus**.

Formats supportés

- **FITS** : .fits, .fit, .fts (majuscules/minuscules)
- **FITS compressé** (fpack) : .fits.fz, .fit.fz
- **XISF** (PixInsight) : .xisf

Les images RGB sont automatiquement converties en luminance. Les images supérieures à 4096×4096 sont binnées 2×2 automatiquement.

Principe de fonctionnement

1. Cliquez **Parcourir** pour charger une image, puis **Analyser**.
2. Détection des étoiles (DAOStarFinder) et ajustement d'une gaussienne 2D elliptique sur chacune.
3. Construction d'une **carte FWHM** (surface polynomiale degré 2).
4. Affichage d'un **champ vectoriel** montrant la direction d'allongement de chaque étoile.
5. **Verdict** : Correct, Trop court (ajouter des espaceurs), ou Trop long (retirer des espaceurs).

Mosaïque 3×3

Vue d'inspection en 9 tuiles (coins, bords, centre) avec le FWHM moyen par zone et des bordures colorées par qualité relative au centre :

- **Vert** ($\leq 1.05\times$) — Excellent
- **Cyan** ($\leq 1.20\times$) — Bon
- **Orange** ($\leq 1.40\times$) — Moyen
- **Rouge** ($> 1.40\times$) — Mauvais

Interprétation

Pourquoi ça marche : Quand le backfocus est incorrect, le correcteur/réducteur ne projette pas un plan focal plat sur le capteur. Le défaut de mise au point augmente radialement depuis le centre optique. La *forme* de l'allongement des étoiles révèle le *sens* de l'erreur :

- **Allongement radial** (étoiles pointant vers le bord) : le foyer est en avant du capteur → backfocus trop court → ajouter des espaceurs.
- **Allongement tangentiel** (étoiles perpendiculaires au rayon, motif de tourbillon) : le foyer est en arrière du capteur → backfocus trop long → retirer des espaceurs.
- Si les étoiles sont *rondes* partout : le backfocus est correct (ou l'erreur est trop faible pour être détectée).

L'analyse mono-image donne la *direction* de l'erreur, pas la valeur précise en mm. Pour une mesure quantitative, utilisez **HocusFocus** avec **N.I.N.A.** (Nighttime Imaging 'N' Astronomy), qui effectue des séries d'images défocalisées à différentes positions du focuser pour calculer l'offset exact.

Dépendances

L'analyseur nécessite des paquets Python supplémentaires (numpy, scipy, astropy, photutils, matplotlib). Ils sont installés automatiquement par `launch.bat` / `launch.sh`. Sans ces paquets, le reste de l'application fonctionne normalement.

Rapports de bugs et capture de crashes

L'application intègre un système de capture d'erreurs et de rapport de bugs entièrement **anonymisé**.

Capture automatique des crashes

Toutes les exceptions non gérées sont automatiquement interceptées :

- Le type d'erreur, le message et le traceback complet sont enregistrés dans un journal local (`backfocus_errors.log`)
- Un fichier de rapport (`_crash_report.json`) est créé avec les informations système (OS, Python, version, architecture)
- Le journal d'erreurs est automatiquement tronqué s'il dépasse 100 Ko

Anonymisation

Sécurité

Tous les rapports sont **anonymisés** avant envoi : les chemins de fichiers sont nettoyés pour remplacer le répertoire personnel (ex : `C:\Users\nom\...`) par `~\...`. Aucun nom d'utilisateur n'apparaît dans les rapports.

Détection au redémarrage

Au prochain lancement après un crash, l'application détecte le fichier de rapport et propose :

- **Envoyer le rapport** : ouvre votre navigateur avec une Issue GitHub pré-remplie contenant le traceback, les informations système et les dernières erreurs du journal
- **Ignorer** : supprime le rapport sans rien envoyer

Le fichier de rapport est supprimé dans les deux cas. **Aucune donnée n'est envoyée automatiquement** — vous devez cliquer « Submit » sur GitHub pour créer l'Issue.

Rapport de bug manuel

Le menu **Aide > Signaler un bug** ouvre le navigateur avec un formulaire d'Issue GitHub pré-rempli. Le rapport inclut automatiquement :

- Les informations système (version de l'application, OS, Python, architecture)
- Les 10 dernières erreurs du journal (si disponibles)
- Tous les chemins sont anonymisés

Mises à jour automatiques

L'application vérifie automatiquement les mises à jour au démarrage via l'API GitHub Releases.

Vérification automatique

- Vérification silencieuse en arrière-plan (thread dédié, 2 secondes après le lancement)
- Si une nouvelle version est disponible, un dialogue affiche le changelog et propose le téléchargement

Vérification manuelle

Menu **Aide > Vérifier les mises à jour**. Affiche un message si aucune mise à jour n'est disponible.

Sécurité du téléchargement

- Limite de taille du téléchargement (50 Mo max)
- Validation de l'archive zip (anti path-traversal, anti symlink)
- Whitelist de fichiers à remplacer (jamais les données utilisateur, le `venv`, `.git`)
- Redémarrage automatique après mise à jour

Raccourci bureau et multi-PC

Création du raccourci

Menu **Aide > Créer un raccourci bureau**. Fonctionne sur Windows, Linux et macOS.

- Le raccourci cible `launch.bat` / `launch.sh` (portable entre PCs)
- L'icône est copiée localement pour éviter les problèmes de chemins réseau
- Au premier lancement, l'application propose de créer le raccourci

Architecture multi-PC / NAS

- Le dossier projet peut être sur un NAS ou un disque synchronisé
- L'environnement virtuel (venv) est créé **localement sur chaque PC** :
 - Windows : `%LOCALAPPDATA%\BackfocusCalculator\venv`
 - Linux : `~/.local/share/BackfocusCalculator/venv`
 - macOS : `~/.local/share/BackfocusCalculator/venv`
- Le venv est validé au lancement (recréé si corrompu ou provenant d'un autre PC)
- Les dépendances sont réinstallées uniquement si `requirements.txt` a changé

Performance et fluidité (v2.0)

La version 2.0 migre l'interface de tkinter vers **PyQt6** et apporte des améliorations significatives de performance :

Interface PyQt6 native

L'interface est entièrement réécrite en PyQt6 avec un style QSS intégré, éliminant les limitations de tkinter/Tcl. Le rendu est plus fluide et le thème sombre est appliqué de manière plus cohérente.

Sauvegarde asynchrone

La sérialisation JSON reste dans le thread principal, mais l'écriture sur disque (write + fsync + replace) est déportée sur un thread dédié en arrière-plan (`_AsyncSaveWriter`). Cela élimine les micro-blocages de 50–200 ms qui survenaient à chaque sauvegarde. Les chemins critiques (fermeture, redémarrage, export) utilisent toujours une sauvegarde synchrone.

Curseur galaxie optimisé

Le taux de rafraîchissement du curseur galaxie est de 30 fps avec un mécanisme « early-out » qui saute le repositionnement si la position n'a pas changé.

Catalogue : mise à jour rapide des quantités

Les boutons `+/-` du catalogue mettent à jour uniquement la ligne concernée au lieu de régénérer la liste entière.

Cache de recherche pièces

Un index pré-calculé évite de reconcaténer 12 000+ chaînes à chaque frappe dans la recherche. Le cache est invalidé automatiquement lors des modifications.

Débounce unifié

Un seul timer de 300 ms pour toutes les opérations de sauvegarde, éliminant les risques de double écriture.

Part II

English Manual

Introduction

The **Backfocus Calculator v2.0** is a desktop graphical application built with **PyQt6** for calculating and managing backfocus configurations in astrophotography optical trains.

What is Backfocus?

In astrophotography, **backfocus** is the precise optical distance required between the last optical correcting element (flattener, focal reducer, coma corrector) and the camera sensor. Incorrect backfocus causes:

- Deformed stars (coma, astigmatism)
- Curved field (sharp center, soft edges)
- Optical tilt
- Asymmetric vignetting

Each corrector/reducer is designed for a specific backfocus distance (typically 55 mm, but ranging from 17 mm to 105 mm).

Optical Convention

Light travels from **left to right** throughout the application:

- **Telescope side** (left): the side through which light enters
- **Camera side** (right): the side through which light exits toward the sensor

Each piece has a **telescope-side** connection and a **camera-side** connection, each with a thread diameter and gender (male = external, female = internal).

Key Features

- Built-in reference database of **12,000+ real products** (125 brands)
- Parts catalog with **22 equipment types**
- **Auto-fill** fields from the reference database
- Build optical trains with **connection compatibility** verification
- User-defined backfocus zone (start/end markers)
- **Flip** reversible pieces (swap T-side/C-side)
- **Suggest** and **combinatorial auto-complete** (1 to 3 parts)
- **Conflict detection** based on owned quantities
- Color-coded visual diagram with highlighted backfocus zone
- **29 connection types**: M42–M117, EOS, Canon RF, Nikon F/Z, Sony E, Fuji X, MFT, SC, ZWO/QHY bolt mounts
- Measurement units: mm/inches, g/ounces
- **Auto-save** on close
- **Export/Import** individual configurations or **all data**
- Bilingual English/French interface
- Dark space theme with animated galaxy cursor
- **Automatic updates** via GitHub Releases
- **Anonymous bug reports** (auto crash capture + manual report)
- **Desktop shortcut** created from Help menu

Installation

Requirements

- **Python 3.8** or higher
- **PyQt6** (installed automatically by the launcher scripts)
- **No manual installation** required

Windows

Double-click `launch.bat`. The script will:

1. Detect Python on your system
2. Create a **local** virtual environment (in `%LOCALAPPDATA%\BackfocusCalculator\venv`)
3. Install all dependencies from `requirements.txt`
4. Launch the application

Multi-PC / NAS

The virtual environment is stored **locally on each PC**, not in the project folder. The project folder can live on a NAS or synced drive without any conflicts.

Linux

Debian/Ubuntu

```
sudo apt-get install python3 python3-venv python3-pip
```

Fedora

```
sudo dnf install python3 python3-pip
```

Arch

```
sudo pacman -S python
```

Launch

```
chmod +x launch.sh && ./launch.sh
```

The venv is created in `~/.local/share/BackfocusCalculator/venv`.

macOS

```
brew install python
```

```
chmod +x launch.sh && ./launch.sh
```

Direct Launch

```
pip install PyQt6
```

```
python backfocus.py
```

Windows Standalone (.exe) — No Python Required

A pre-built Windows executable is available on the [Releases](#) page:

1. Download the latest `BackfocusCalculator-vX.X.X.zip`
2. Extract the archive
3. Run `BackfocusCalculator.exe` — no Python installation needed

To build the executable yourself:

```
pip install pyinstaller
pyinstaller backfocus.spec
```

The output is in `dist/BackfocusCalculator/`. All features work identically.

Note: the standalone exe is Windows-only. On Linux/macOS, use the launcher scripts above.

Parts Catalog

The catalog opens in a **large separate window** via the **View > Parts Catalog** menu or the toolbar button.

First Launch

On first launch, the full 12,000+ product database is automatically loaded into your catalog with an owned quantity of 0.

Adding a Part

Click **Add** and fill in:

- **Brand** — dropdown with 125+ brands + “Custom Made” option
- **Name** — product name
- **Type** — from 22 categories (see Section 25)
- **Optical length** — contribution to backfocus in mm
- **Mass** — in grams
- **Telescope-side connection** — thread + gender (Male/Female)
- **Camera-side connection** — thread + gender
- **Reversible** — check if the part can be flipped
- **Backfocus role** — **start**, **end**, or empty
- **Quantity** — number of copies owned
- **Notes** — free text

Auto-fill from Reference Database

1. Type the beginning of a brand or product name
2. Click the **Auto-fill** button
3. All fields are populated automatically from the 12,000+ product database

Editing, Duplicating, Deleting

- **Double-click** a part to edit it
- **Duplicate** creates a copy (“(copy)” suffix)
- **Delete** asks for confirmation

Owned Quantity

The + and – buttons quickly adjust quantity:

- Quantity > 0 ⇒ part displayed in **gold** (owned)
- Quantity = 0 ⇒ part displayed in **gray** (not owned)

Advanced Filters

Five combinable filters (AND logic):

1. **Text** — searches in brand and name
2. **Type** — filter by category

3. **Diameter** — filter by thread (M42, M48, M54, M68, SC, EOS, etc.)
4. **Gender** — Male, Female, or All
5. **Owned only** — check to show only owned parts

Reset button to clear all filters.

Column Sorting

Click any column header to sort. Second click reverses the order.

Configurations

The main window is dedicated to optical train configurations.

Creating a Configuration

1. Click **+** in the left panel
2. Give it a name (e.g., “My Newton + ASI 2600”)
3. Set the **target backfocus** in mm (e.g., 55)
4. Add **notes** if needed

Managing Configurations

- **-**: delete (with confirmation)
- **Rename**: change the name
- **Duplicate**: create a full copy

Building the Optical Train

1. Click **Add part** to choose from the catalog
2. Parts are added in telescope → camera order
3. **Up/Down**: reorder parts
4. **Flip**: reverse a reversible part (swaps T-side/C-side connections)
5. **Remove**: take a part out of the train

Ghost Pieces

When you add a part whose connections are incompatible with the previous part (e.g., an M48 thread after an M42), the application offers several options:

- **Flip and insert**: flips the part if it is reversible
- **Insert anyway**: ignores the incompatibility
- **Insert placeholder**: inserts a *ghost adapter* between the two parts
- **Edit part**: opens the editor to fix connections

The ghost appears in **orange** in the train with the name “? Missing adapter”. It represents an adapter ring you need to find. Its connections are automatically inferred from the neighboring parts.

To resolve a ghost:

1. Click **Resolve ghosts** in the toolbar
2. The application searches your catalog for parts matching the required connections
3. Select a part and confirm: it **replaces the ghost** at its exact position in the train

You can also manually insert a ghost using the **Insert ghost** button to mark a slot to be filled.

Backfocus Zone

1. Select the “start” part (e.g., reducer) → **Set BF start**
2. Select the “end” part (e.g., camera) → **Set BF end**
3. Calculated BF = sum of optical lengths **after** the start marker up to the end marker (the start piece is the zero point, its length is not counted)

Automatic Calculation

The application calculates in real time:

- **Total**: total optical length of the train
- **Backfocus**: length within the BF zone only
- **Gap**:
 - **Green / OK**: gap < 0.1 mm
 - **Orange / Short**: backfocus insufficient
 - **Red / Long**: backfocus excessive
- **Compatibility**: checks between each pair of adjacent parts
- **Conflicts**: warns if a part exceeds owned quantity

Simple Suggestion

Suggest part: finds ONE owned part that would fill the remaining gap. Checks connection compatibility. Results sorted by proximity.

Combinatorial Auto-complete

Auto-complete: searches among combinations of 1, 2, or 3 owned parts. Option to allow/disallow parts from other configurations. Perfect solutions in **green**.

Visual Diagram

Diagram at the bottom: telescope on the left, camera on the right. Each color corresponds to a part type. BF zone highlighted. Dashed red line = target. Drag and drop to reorder parts.

Save and Export

Automatic Save

- All data is saved in `backfocus_data.json` on close
- **File > Save All** to save manually at any time

Configuration Export / Import

- **File > Export Configuration** — saves active config as JSON
- **File > Import Configuration** — loads a config from JSON

Full Data Export / Import

- **File > Export All Data** — saves EVERYTHING: parts, configs, settings
- **File > Import All Data** — replaces ALL your data (with confirmation)
- Ideal for transferring between computers or complete backup

Measurement Units

Settings > **Measurement Units** menu:

- Lengths: **mm** or **inches**
- Mass: **grams** or **ounces**

Keyboard Shortcuts

Shortcut	Action
Enter	Confirm in all dialog windows
Escape	Close / cancel all dialog windows
Double-click (catalog)	Edit the selected part
Double-click (suggestions)	Insert the part or solution

Part Types

Internal key	English	BF role
type_telescope	Telescope	—
type_refractor	Refractor	—
type_camera_lens	Camera Lens	—
type_camera	Astro Camera	<i>end</i>
type_dslr	DSLR / Mirrorless	<i>end</i>
type_eyepiece	Eyepiece	<i>end</i>
type_barlow	Barlow Lens	<i>start</i>
type_reducer	Focal Reducer	<i>start</i>
type_flattener	Field Flattener	<i>start</i>
type_extender	Focal Extender	<i>start</i>
type_corrector	Coma Corrector	<i>start</i>
type_filter_wheel	Filter Wheel	—
type_filter_holder	Filter Holder	—
type_oag	OAG	—
type_rotator	Rotator	—
type_focuser	Focuser	—
type_diagonal	Diagonal	—
type_adapter	Adapter Ring	—
type_spacer	Spacer	—
type_anti_tilt	Anti-tilt Adapter	—
type_guide_scope	Guide Scope	—
type_flip_mirror	Flip Mirror	—

Connection Types

Type	Description	Diameter (mm)
M42 / T2	Standard T2 thread (M42×0.75)	42
M48	2" filter thread	48
M54	ZWO / common camera thread	54
M56	Specific camera thread	56
M63	Medium adapter thread	63
M68	Large format adapter thread	68
M72	Flattener/reducer thread	72
M81	Large focuser thread	81
M82	Very large thread	82
M84	Extra-large thread	84
M92	Specialized thread	92
M117	Very large format thread	117
SC	Schmidt-Cassegrain bayonet	—
EOS	Canon EOS (EF) mount	54
Canon RF	Canon RF mount	54
Nikon F	Nikon F mount	44
Nikon Z	Nikon Z mount	55
Sony E	Sony E mount	46
Fuji X	Fujifilm X mount	44
MFT	Micro Four Thirds mount	38
Pentax K	Pentax K mount	44
CS	C/CS mount	25.4
1.25"	1.25" eyepiece barrel	31.75
2"	2" eyepiece barrel	50.8
ZWO 6-bolt	ZWO 6-screw attachment	—
ZWO 4-bolt	ZWO 4-screw attachment	—
QHY 4-bolt	QHY 4-screw attachment	—

Interface and Theme

Dark Space Theme

Dark theme inspired by space: black/dark blue background with colorful accents. Owned parts in gold, non-owned in gray. The interface is fully built with **PyQt6** and an integrated QSS stylesheet.

Galaxy Cursor

A spiral galaxy cursor follows your mouse. Purely decorative.

Language Switching

Language menu → **Français** or **English**. Instant and remembered.

Project Structure

backfocus/

- backfocus.py	Main application (PyQt6, ~3400 lines)
- reference_data.py	Reference database (12,100+ entries)
- gen_refdb.py	Database generator script
- backfocus_data.json	User data (auto-created)
- fits_analyzer.py	FITS/XISF backfocus analyzer (optional)
- test_audit.py	Automated test suite
- shortcut_helper.py	Desktop shortcut creation helper
- launch.bat	Windows launcher (local venv)
- launch.sh	Linux/macOS launcher (local venv)
- requirements.txt	Python dependencies (PyQt6 + FITS analyzer)
- backfocus.spec	PyInstaller build spec (Windows exe)
- backfocus.ico	Application icon (Windows)
- backfocus.png	Application icon (cross-platform)
- README.md	GitHub documentation
- manual/	
' - manual.pdf	Bilingual user manual (PDF)

The Backfocus Concept

Backfocus is measured from the camera-side of the corrector/reducer/flattener to the sensor.

Common backfocus examples:

Corrector / Reducer	Required BF
Sky-Watcher 0.85x Reducer	55 mm
Baader MPCC Mark III	55 mm
Celestron f/6.3 Reducer	99 mm
Starizona HyperStar	12–24 mm
Takahashi TOA-35 Reducer	72.2 mm
Riccardi 0.75x M82 Reducer	56 mm
ASA 3" Corrector	67 mm

Some telescope designs (e.g., Petzval refractors like the Takahashi FSQ-106ED) do not have a separate backfocus concept. The application supports these by simply not defining a BF zone.

FITS / XISF Backfocus Analyzer

The application includes an image analyzer that diagnoses backfocus errors from a single astro image. Open it via **View > FITS / XISF Backfocus Analyzer**.

Supported Formats

- **FITS**: .fits, .fit, .fts (case-insensitive)
- **Compressed FITS** (fpack): .fits.fz, .fit.fz
- **XISF** (PixInsight): .xisf

RGB images are automatically converted to luminance. Images larger than 4096×4096 are automatically binned 2×2.

How It Works

1. Click **Browse** to load an image, then **Analyze**.
2. Star detection (DAOStarFinder) and elliptical 2D Gaussian fitting on each star.

3. A **FWHM map** is built (degree-2 polynomial surface).
4. A **vector field** shows the elongation direction of each star.
5. **Verdict**: Correct, Too short (add spacers), or Too long (remove spacers).

Mosaic 3×3

Inspection view with 9 tiles (corners, edges, center) showing mean FWHM per zone and color-coded borders by quality relative to center:

- **Green** ($\leq 1.05\times$) — Excellent
- **Teal** ($\leq 1.20\times$) — Good
- **Orange** ($\leq 1.40\times$) — Fair
- **Red** ($> 1.40\times$) — Poor

Interpretation

Why it works: When backfocus is incorrect, the corrector/reducer does not project a flat focal plane onto the sensor. The focus error increases radially from the optical center. The *shape* of star elongation reveals the *direction* of the error:

- **Radial elongation** (stars pointing toward the edge): the focal plane is in front of the sensor → backfocus too short → add spacers.
- **Tangential elongation** (stars perpendicular to radius, swirl pattern): the focal plane is behind the sensor → backfocus too long → remove spacers.
- If stars are *round* everywhere: backfocus is correct (or the error is too small to detect).

Single-image analysis gives the *direction* of the error, not the precise value in mm. For quantitative measurement, use **HocusFocus** with **N.I.N.A.** (Nighttime Imaging 'N' Astronomy), which takes defocused image series at different focuser positions to compute the exact offset.

Dependencies

The analyzer requires additional Python packages (numpy, scipy, astropy, photutils, matplotlib). These are installed automatically by `launch.bat` / `launch.sh`. Without these packages, the rest of the application works normally.

Bug Reports and Crash Capture

The application includes a built-in error capture and bug reporting system that is fully **anonymized**.

Automatic Crash Capture

All unhandled exceptions are automatically intercepted:

- The error type, message, and full traceback are logged to a local file (`backfocus_errors.log`)
- A crash report file (`_crash_report.json`) is created with system information (OS, Python, version, architecture)
- The error log is automatically truncated if it exceeds 100 KB

Anonymization

Security

All reports are **anonymized** before submission: file paths are sanitized to replace the user home directory (e.g., `C:\Users\name\...`) with `~\...`. No username ever appears in the reports.

Crash Detection on Restart

On the next launch after a crash, the application detects the report file and offers:

- **Send Report**: opens your browser with a pre-filled GitHub Issue containing the traceback, system information, and recent errors from the log
- **Skip**: deletes the report without sending anything

The report file is deleted in both cases. **No data is sent automatically** — you must click “Submit” on GitHub to create the Issue.

Manual Bug Report

The **Help > Report Bug** menu opens your browser with a pre-filled GitHub Issue form. The report automatically includes:

- System information (application version, OS, Python, architecture)
- The last 10 errors from the log (if available)
- All paths are anonymized

Automatic Updates

The application automatically checks for updates on startup via the GitHub Releases API.

Automatic Check

- Silent background check (dedicated thread, 2 seconds after launch)
- If a new version is available, a dialog shows the changelog and offers to download

Manual Check

Help > Check for Updates menu. Shows a message if no update is available.

Download Security

- Download size limit (50 MB max)
- Zip archive validation (anti path-traversal, anti symlink)
- Whitelist of files to replace (never touches user data, venv, .git)
- Automatic restart after update

Desktop Shortcut and Multi-PC

Creating the Shortcut

Help > Create Desktop Shortcut menu. Works on Windows, Linux, and macOS.

- The shortcut targets `launch.bat` / `launch.sh` (portable across PCs)
- The icon is copied locally to avoid network path issues
- On first launch, the application offers to create the shortcut

Multi-PC / NAS Architecture

- The project folder can live on a NAS or synced drive
- The virtual environment (venv) is created **locally on each PC**:
 - Windows: %LOCALAPPDATA%\BackfocusCalculator\venv
 - Linux: ~/.local/share/BackfocusCalculator/venv
 - macOS: ~/.local/share/BackfocusCalculator/venv
- The venv is validated on launch (recreated if corrupt or from another PC)
- Dependencies are reinstalled only if `requirements.txt` has changed

Performance & Fluidity (v2.0)

Version 2.0 migrates the interface from tkinter to **PyQt6** and brings significant performance improvements:

Native PyQt6 Interface

The interface is fully rewritten in PyQt6 with an integrated QSS stylesheet, eliminating tkinter/Tcl limitations. Rendering is smoother and the dark theme is applied more consistently.

Asynchronous Save

JSON serialization still happens in the main thread, but file I/O (write + fsync + replace) is offloaded to a dedicated background daemon thread (`_AsyncSaveWriter`). This eliminates the 50–200 ms UI micro-freezes that occurred on every save. Critical paths (close, restart, export) still use synchronous save.

Galaxy Cursor Optimization

The galaxy cursor polls at 30 fps with an early-out mechanism that skips repositioning when the rendered position hasn't changed.

Catalog: Fast Quantity Update

The +/- buttons in the catalog update only the affected row instead of rebuilding the entire list.

Parts Search Cache

A pre-built index avoids re-concatenating 12,000+ strings on each keystroke in the part picker search. The cache is automatically invalidated on data changes.

Unified Debounce

A single 300 ms timer for all save operations, eliminating potential double-writes.